Table of Contents

Section Title	Page Number
Virtual Machine : The Infrastructure Layer :	1
Inside the Infrastructure Layer :	1
Working of Infrastructure Layer :	2
Virtual Machine : Hypervisor Layer :	2
Inside the Hypervisor Layer : Virtual Machine :	3
Working of Hypervisor Layer : Virtual Machine :	3
So first : (Hardware Abstraction / Abstraction) :	4
Second : (Isolation) :	4
Third : (Resource Scheduling / Partitioning) :	4
Fourth : (Emulation) :	5
In Virtual Machine : App1 Layer :	5
Inside the App1 Layer : Based on actual software you want to use :	5

Based on building a web service :	6
Based on Software Functionality :	6
Working of App1 : Virtual Machine :	6
Isolation :	7
Self-Contained :	7
Virtual Machine : Guest OS Layer :	7
Inside the Guest OS layer : Virtual Machine :	8
Kernel :	8
Device Drivers :	8
System Libraries :	9
File System :	9
System Services :	9
Applications :	9
Each VM need its own Guest OS :	10

Working of Guest OS : Virtual Machine :	10
Therefore,	10
Virtual Machine : Bins / Lib Layer :	11
Inside : Binaries (Bins) :	11
Inside : Libraries (Lib) :	11
Importance : Libraries and Binaries Layer : Virtual Machine :	12
Working of Bins/Lib : Virtual machine :	13
Containers : Infrastructure Layer :	14
Containers : Infrastructure Layer : Inside :	14
Containers : Infrastructure Layer : Working :	15
Containers : Operating System Layer :	16
Containers : Operating System Layers : Inside :	16
Containers : Operating System Layer : Working :	17
Containers : Container Engine Layer :	18

Containers : Container Engine Layer : Inside :	18
Inside : Container Engine Layer : Containers :	19
Container Runtime :	19
Image Management :	19
APIs and CLI :	19
Resource Management :	19
Networking Setup :	19
Storage Management :	20
Therefore,	20
Container : Container Engine Layer : Working :	20
Therefore,	21
Containers : App1 : Layer :	21
Containers : App1 : Inside :	21
Containers : App1 : Working :	22

Comparison between Virtual Machines and Containers	22
Working of Virtual Machines and Containers :	23
Infrastructure Layer :	23
Hypervisor Layer :	23
Guest OS Layers :	23
Host OS Layers :	23
Container Engine Layer :	23
Bins/Lib Layer :	23
Application Layer :	24
Key points : Virtual Machine and Container :	24
Virtual Machine :	24
Containers :	24
Examples : Virtual Machine :	24
Containers :	24

Comparison between Virtual Machine Layers and Containers Layers : 25

Infrastructure Layer (The Physical Hardware): 25

Hypervisor Layer (Only in Virtual Machine): 25

Guest OS Layer (Only in VMs) : 26

Container Engine Layer (Only in Containers) : 26

Bins/Libs Layer : 27

App Layer : 27

One powerful machine are run by these Virtual Machine and Containers : 28

Using Virtual Machines : 28

Using Containers : 28

Name : AKASH SAMPATH

Topic : Virtualization and Containerization

Virtual Machine : The Infrastructure Layer :

- The infrastructure layer is the physical foundation that supports everything above it and it is the actual hardware with basic facilities that provide raw computing power , storage and network connectivity
- These are physical components where you can touch and interact and without this layer, the hypervisor , Guest OS and Apps wont exists and its like a software without a computer
- This layer is the actual physical machine and the foundation that provides everything the Virtual Machine need to exist

Inside the Infrastructure Layer :

- We have Physical Servers such as actual computers with CPU , RAM , Local disk and we have storage systems such as hard drive and networking systems such as switches , routers , cables and network interface which used for internet connectivity
- There are some main physical components in this layer such as the CPU where this does the processing and RAM is the short term memory where the computer keeps data it needs to access right now and the Storage is where al the data , OS files and apps are saved permanently and Networking components that allows the hardware to talk to the internet

Working of Infrastructure Layer :

- For Virtual Machine the hypervisor sits directly on the infrastructure layer and divides those physical resources into virtual ones allowing multiple guest OS to share the same hardware
- In Virtual Machine setup the hardware doesn't know its running multiple apps or operating systems
- And therefore the infrastructure is the bottom layer that delivers CPU , Memory and disk space and network connection to all the software layers above and with is layer the entire stack is built

Virtual Machine : Hypervisor Layer :

- The hypervisor is a software layer that sits between the physical hardware (infrastructure layer) and the virtual machines and its job is to filter (abstract) the hardware so that multiple operating systems can run on the same physical machine and each thinking it has its own dedicated hardware
- Also the hypervisor is known as a Virtual Machine Monitor which is the brain of the virtual machines (Virtualization) and it acts as a middle men between the physical hardware (infrastructure) and the multiple operating systems (Guest OS)
- This is also a software layer that sits directly on the physical hardware and tricks the operating system thinking they each have their own computer

Inside the Hypervisor Layer : Virtual Machine :

- Hypervisor manages the CPU and since multiple virtual machines want to use the processor at the same time , the hypervisor decides which VM gets to use the CPU for a few seconds and then swaps in the next one and ensures each Virtual Machines doesn't steal all the CPU from each other

- Hypervisor connects (maps) Virtual RAM to Physical RAM by ensuring that app1 cannot see or overwrite the memory being uses by app2

- The hypervisor mimics hardware components so that the guest OS can use them without knowing they are virtual and making each Virtual machine thinking it is real computer

Working of Hypervisor Layer : Virtual Machine :

- A Hypervisor sits between the physical hardware like computer's CPU , Memory and Storage and the virtual machines and shares the physical hardware resources among multiple Virtual Machines and making each VM think it has its own hardware
- Also it takes one physical machine (Infrastructure Layer) and makes it think it is independent computer
- It also sits between the physical hardware and the multiple virtual machines
- This also manages resources so multiple operating systems (Guest) can share one physical computer (Host)

So first : (Hardware Abstraction / Abstraction) :

- The hypervisor takes the real physical hardware like CPU crores , RAM , disk , and network components and creates a virtual versions of these components and also creating a fake virtual versions of these hardware parts for each VM and each VM thinks it has its own processor and memory and but it's just a small piece of real physical hardware
- Create 4 virtual CPUs from 2 physical CPUs and take 8GB of virtual RAM from 16GB physical RAM

Second : (Isolation) :

- Each VM runs its own operating system (Guest OS) and applications and the hypervisor ensures they don't interfere with each other and if one VM crashes the others keep running
- This is like a security wall and if app1 crashes or gets a virus , the hypervisor ensures it stays trapped inside its own box (app1) and it cannot reach over and affect app2 because the hypervisor keeps them completely separated
- As the hypervisor keeps each VM in its own space and if app1 or its guest OS crashes and the hypervisor ensures it doesn't affect app2 or the rest of the system

Third : (Resource Scheduling / Partitioning) :

- The hypervisor acts like a schedular when a VM needs to do some work like run a program and the hypervisor decides when and how much of the Physical CPU and memory to give
- It might give some CPU time to first VM and then switch to second VM and so on and it feels like they are all running at once
- Its uses the physical hardware and if your laptop has 16GB or RAM , the hypervisor might give 4GB to first VM (Guest OS 1) and 8GB to second VM (Guest OS 2) and ensures they don't fight over the same resources

Fourth : (Emulation) :

- The hypervisor creates a virtual hardware and the guest OS thinks is seeing a real hard drive and a real network component and actually they are just software files managed by the hypervisor so that the Guest OS doesn't need special drivers of software

In Virtual Machine : App1 Layer :

- In virtual machine the app1 layer represents a single application like a web server , database , or a custom program that is running inside that virtual machine and it's the actual software you want to use
- App1 layer is the top most level where your actual software lives and sitting directly on top of the libraries and the operating system used to that specific VM

Inside the App1 Layer : Based on actual software you want to use :

- Application code is the actual program files like python scripts and .exe files , java files
- Configuration files are the settings that tell the app how to behave
- Runtime data is temporary files or logs the app creates while running
- Dependencies are where the app needs specific libraries or framework but those are usually stored in the Bins/Lib layer

Based on building a web service :

- Source code is the complied code that runs your logic like .jar file for java and .py file for python or .exe for Windows
- Applications Configs are files like application.properties , .env file or XML configurations that tell the app which database to connect to and which port to use
- Static Assets are in the web app contains HTML , CSS , images and JavaScript files
- Local Data is where any temporary files or logs the application generates while its running

Based on Software Functionality :

- Source code is the actual logic and instruction of the program
- Application Configuration is the settings , environment variables and parameters that tell the app how to behave
- User Interface Components is when if it's a user facing app , the code that generates what you see on the screen

Working of App1 : Virtual Machine :

App1 runs on top of a Guest OS :

- Inside the VM , there is a full operating system like Windows or Linux called the Guest OS .
- The app is installed and runs just like it would on a physical computer it relies (depends) on the Guest OS to provide services like accessing files or using memory and communication through network

The Guest OS thinks its on real hardware :

- The Guest OS doesn't know its virtualized and it communicates with what it believes is the physical hardware like CPU , RAM and disk but actually those are virtual devices provided by the hypervisor

Isolation :

- App1 is completely isolated from other apps like app2 and app3 that might be running in separate VMs on the same physical machine
- Each VM has its own Guest OS and its own copy of libraries and binaries so app1 doesn't interfere with others

Self-Contained :

- When app1 runs it believes it has the entire computer itself and doesn't know that app2 or app3 exist
- This is because the Guest OS (the operating system inside the specific VM) provides a private environment for it

Virtual Machine : Guest OS Layer :

- In a virtual machine the Guest OS layer is the operating system that runs inside the virtual machine just like the OS on a physical computer it is called a guest
- Its running on top of a hypervisor which manages the hardware resources
- The Guest OS layer is what makes the VM feel like a completely separate computer while the hypervisor mimics the physical hardware like fake CPU , RAM
- The Guest OS is the software that loves top of that fake hardware
- The Guest OS acts like a normal operating system like Windows or Linux installed on a physical laptop but it thinks the virtual hardware provided by the hypervisor is real

Inside the Guest OS layer : Virtual Machine :

- The guest OS is a complete operating system and it contains all the core components you find in any OS installed on a real machine

Kernel :

- The heart of the OS and it manages the CPU , Memory and devices. In a Virtual machine , the kernel still thinks its talking to real hardware but the hypervisor gets the request and translates them to the physical hardware
- The kernel is the core of any operating system and its is the first thing to load and the last thing to shut down
- The kernel manages all the fake resources provided by the hypervisor and it decides which application gets to use the CPU and how much memory is allocated to a specific task
- In virtual machine the guest OS kernel has no idea its running on a hypervisor and thinks it is talking directly
- Being the brain of the Guest OS , it manages the virtual machine virtual CPU , Virtual memory and internal processes

Device Drivers :

- These are special programs that allow the OS to communicate with virtual hardware like virtual network , disk , and virtual CPU
- The hypervisor presents virtual devices that look like real one and so the guest OS load drivers for them like it would on a physical PC
- Even though the hardware is virtual, the OS still needs drivers to talk to it
- These are specialized pieces of software that translates something the virtual hardware understands
- Special software that allows the Guest OS to talk to fake virtual hardware such as virtual network or hard disks

System Libraries :

- Shared code like DLL on Windows that applications rely on to perform basic tasks like reading files , opening network connections and etc
- These are files like .dll in windows that contain standard instructions for common tasks like drawing a window on the screen and sending file over a network or encrypting data
- These are files that provide the basic functions and tools needed for applications to run

File System :

- The guest OS has its own file system and stored in a virtual disk file and it contains the OS files , applications and user data
- The guest OS manages its own disk space and it organizes how data is stored on the virtual hard drive
- It creates the folder structure as C:\ or /home/user and keeps track of where every bit of data is located
- The Guest OS has its own virtual hard drive where it stores its own system files , user data and configurations

System Services :

- These are programs that start automatically and run in the background without you seeing them
- With networking it keeps the VM connected to the internet and with logging it keeps the record of everything that happens in the system and the security handles logins and user permissions

Applications :

- The actual programs you want to run like web server , database or custom app and they sit on top of the OS and thus uses its kernel

Each VM need its own Guest OS :

- Hypervisor creates an isolated virtualized hardware environment and each virtual machine mush have a full OS to manage the environment
- In Virtual Machines each VM has a complete OS and provides isolation and lets you run different OS like Windows on a Linux Host
- The Guest OS is completely separated from the Host OS and other VMs and if one Guest OS crashes or gets virus and won't affect others
- You can run a Linux Guest OS on a Windows Host or vice – versa

Working of Guest OS : Virtual Machine :

- Inside the Virtual machine known as VM , The Guest OS layer works like a complete operating system running inside a computer that it itself inside a another computer
- The Guest OS is the operating system installed inside the virtual machine and just like you install Windows or Linux on a real computer
- Like installing Ubuntu or Windows inside a VM
- This OS does not know it is virtual and behaves like it running on real hardware
- First an application inside the VM starts like Browser or Python program of web application or server
- The application needs CPU , Memory and Storage
- Second the Guest OS kernel manages the requests just like a normal OS
- Requests like CPU processing , RAM allocation , Disk read and write and accessing network
- Third instead of real hardware the Guest OS talks to virtual devices provided by the hypervisor
- Virtual hardware such as Virtual CPU , RAM , Storage , Network
- Fourth the hypervisor takes the request from the Guest OS and connects to the actual physical hardware

Therefore ,

- The Guest OS is a full operating system running inside a virtual machine that manages applications and sends hardware requests to the hypervisor which uses the real physical hardware

Virtual Machine : Bins / Lib Layer :

- In virtual machine the Bins / Lib layer sits between the Application and the Guest OS
- Bins are known as Binaries and Lib are known as Libraries
- So bins / lib layer helps run the programs and application and its like a tool that your applications needs to its job or application needs to function

Inside : Binaries (Bins) :

- Binaries are executable programs that run directly on the operating system and also commands that operating system and applications use
- These are the instructions that tell the computer what to do based on language runtimes like :

- /bin/bash is the command shell
- /usr/bin/python is the python interpreter
- /usr/bin/java is java runtime

- So these are commands that the operating system and applications use and when we run these commands or start an application the OS looks in these folders to find the program
- If your application needs python, the python binary will be here

Inside : Libraries (Lib) :

- Libraries are reusable pieces of code that programs use instead of writing everything from scratch
- These are shared code files that programs use to perform common tasks
- These are like tools in a toolbox that multiple apps can use
- If a program needs to read a file , it doesn't write that code from scratch and its uses library that already knows how to do that

Importance : Libraries and Binaries Layer : Virtual Machine :

- Each VM contains its own binaries and libraires and operating system and so every VM is like a full computer
- So that's why VMs are heavier
- Each VM has its own full OS known as Guest OS
- The Bins/Lib layer belongs to that Guest OS
- So Application 1 , 2 and 3 all share the same binaries and libraries inside that VM
- Each Virtual Machine has its own Guest OS and its own Bins/Lib Layer
- Because the Bins/Lib are inside the VM where Application 1 can use an old version of library and while application 2 uses a new libraries on the same physical server or machine
- The application doesn't talk directly to the hardware it talks to the Bins and Libs which then talk to the Guest OS and ensures the application has everything it needs

So therefore ,

- This bins and lib layer contains binaries and shares the libraries that application need to run inside the VM
- These are the collection of programs (binaries) and libraries that help the applications inside the VM to run
- These are part of the guest OS and are shared by all application in that VM
- So different applications need different of versions of own binaries and libraries in a virtual machine to avoid interference with other applications

Working of Bins/Lib : Virtual machine :

- The calculator is the application app and calculator.exe is the main program known as binaries
- The files that help it with display UI and read input or do math is known as Libraries
- In virtual machines each VM has separate Bins/Lib and this makes VMs heavy because of libraries are duplicated
- So with 3GB Each Virtual machine gets each libraries GB
- Inside the VM , this layer acts as the middleman between your application and the Guest Operating System
- So in our physical computer there are many application that share the same library files and a Virtual machine gives the application 1 its own copy of every library it needs
- If application 1 needs an old versions of a file and application 2 needs a new version , they don't crash each other because they have their own Bins/Libs layer sitting on their own Guest OS
- When the application wants to do something like save a file , it sends a request to its specific Bins/Lib
- Those libraries then talk to the Guest OS and passes the request down to the hypervisor and the physical hardware or machine
- Also when you open an application and instead of writing everything from scratch , the application calls a library function where the library already know how to handle the application
- The library then asks the Guest OS to perform task like Opening the file and reading the file and writing output and allocating memory
- The Guest OS thinks it owns the hardware but the hypervisor translates the request to real hardware

Containers : Infrastructure Layer :

- The infrastructure layer in containers is the physical or virtual foundation which everything else runs
- It also the actual hardware or the virtualized hardware that provides all raw physical resources
- Also it's the actual hardware resources required to run software
- As containers are lightweight and virtual , they need physical components to make use of containers
- This infrastructure layer is shared by all containers through the Operating system and the Container Engine
- In the this layer the applications inside the containers don't need to know information of hardware and they just trust the infrastructure need to function

Containers : Infrastructure Layer : Inside :

- The Physical Servers such as CPU is the actual computer that perform calculations and it is the brain of the computer
- In container environment the container engine known as Docker tells the OS how to allocate of spilt CPU power to application 1 so that the application 1 doesn't use power from application 2
- The Physical Memory such as RAM is the actual memory that provide fast and temporary storage for running programs
- The data stays while its active and being used and using containers they only take the exact amount of RAM they need from the infrastructure
- The Physical Storage is the actual hard drive where data is stored for long term
- As containers can delete or recreate the storage easily , the infrastructure layer provides the physical space to store container images and logs or data the applications need to remember
- Using the physical networking such as cables , switches or routes that allow data to travel between server such as CPU and to others
- This networking hardware allows the infrastructure layer to send data packets between containers so users can actually access application 1
- Basically the cables and connections that allow containers to talk to the internet or each other

Containers : Infrastructure Layer : Working :

- Basically in containers the infrastructure layer is the operating system
- This layer consist of components such as Physical Hardware and Host Operating System Kernal and these comes along in container model
- Containers run on the physical servers or cloud VMs using raw CPU , RAM and Storage power
- Containers interact with infrastructure layer directly though the Host Operating System
- The infrastructure doesn't know its running a container, it just sees the container engine like Docker asking for resources
- When you start a container the infrastructure layer provides a bit of its CPU and RAM power
- Using Host OS Kernel , Instead of each application having its own Guest OS , Using containers share the Host OS kernel
- Kernel is the core of the operating system and the program that talks to the physical hardware and manages resources like memory and CPU
- Because the kernel is shared the infrastructure layer don't need to boot up operating system for every application
- Using container the infrastructure only supports one Operating system and that OS tells the infrastructure only let application1 use 10 percent of your CPU power
- Using Containers , the infrastructure layer usually runs a single Host OS like Linux or Windows
- The infrastructure layer tricks the processes inside the container into thinking they have the entire machine to themselves even though are all sharing the same kernel

Containers : Operating System Layer :

- Containers don't have their own full operating system and instead they share the host's OS kernel and still they act like separated systems
- Containers don't have their own copy of the operating system and don't need to boot an OS every time they start
- All containers on a host use the same kernel like the core of the operating system like the one that runs on the host machine
- If the host runs Linux all containers also use that Linux kernel
- The operating system separates containers so they don't interfere with each other by isolating processes , files and networks and control how much CPU , Memory and network each container can use
- In containers the operating system layer refers to the Host OS like Linux or Windows Server
- The kernel is the brain of the OS that talks to the physical hardware such as CPU , Memory , Storage
- All containers on a single machine share the same Host OS kernel
- In containers every container uses the same Host OS kernel to communicate with the hardware
- The operating system layer decides how much CPU , Memory , Storage each container gets ensuring no single container gets all the power
- Containers are smaller and uses less memory and starts up in seconds rather than minutes

Containers : Operating System Layers : Inside :

- In container technology like Docker , the operating system layer is shared among all containers running on the same host
- The Linux is known as the Windows Kernel and this is the core of the operating system and manages Memory , CPU and Storage and Networking hardware
- All containers on the same host share the same kernel
- In container there is only one kernel and every container on that machine talks to this same kernel
- If containers share the same OS and to stay private the OS uses a feature called Namespaces

- Namespaces create a private space for each container and inside the space , a container thinks it is the only one running like
- Preventing Application 1 from seeing or stopping the process of application 2 and gives each container its own virtual Ip address and ensures application 1 cant look at application 2 private files
- To keep one container from using other computer power , the operating system layer uses control groups where
- Container A can only use 10 % of the CPU and 512MB of RAM and if a container tries to take more the operating system layer shuts it down to protect the others

Containers : Operating System Layer : Working :

- In containers the host OS kernel is shared but each container gets its own separated space
- Operating system layer is the foundation that makes containers much lighter and containers all share the same Host Operating System kernel
- Containers use the one operating system already running on our physical computer to access applications without creating new operating system for every application
- The host machine has one operating system like Linux , Windows and all containers on that host use the same OS kernel
- The kernel creates separate views for each container using namespaces and each container sees its own files , network connections and others
- Even though there are 50 containers running on the OS and each one thinks it has its own private network and storage
- To the container it looks like it has its own OS , but all in same kernel
- Using control groups we can limit how much CPU , memory or storage each container can use and this prevents one container from using all resources
- If one container starts crashing or overworking the operating system layer ensures it doesn’t crash the whole system

Containers : Container Engine Layer :

- The computer has one main brain like CPU and one main memory like RAM and if you just throw three applications onto the computer , they might fight over resources and crash each other
- Using container engine layer inside the Host OS like Linux or Windows to put each application to work in one computer , This container engine layer tells the computer that application 1 can only use this much CPU power and application 2 can only see this part of the RAM or memory
- This keeps them interfering from each other
- Container Engine is a software that sits on top of the host Operating System and its job is to pack , start and manage containers
- The container engine creates fake operating system environment using the real Host OS
- This container engine layer pulls the necessary application files from registry and unpacks them so they are ready to run

Containers : Container Engine Layer : Inside :

- Container Engine is the brain and manages what makes the containers work and it's the software that creates , runs and manages containers on your system
- As the operating system handles the hardware , the container engine layer manages how your applications run without bumping into each other
- A container engine is like a docker that sits between your applications and the computer operating system
- It is responsible for starting , stopping and running containers by directly talking to the host OS kernel

Inside : Container Engine Layer : Containers :

Container Runtime :

- Core part that actually starts and stops containers and talks to the operating system kernel like Linux to create separated environments for each applications
- Container engine just tells the main operating system to start a new process
- Actually translates your commands like docker run into instructions the system kernel understands to start the application faster

Image Management :

- The engine handles downloading , storing and managing container images and it knows how to pull images from docker hub which is the registries and keep them ready to run
- Before an application can run , the engine needs to find the container image which is snapshot of your applications , its libraires and its settings
- So this pulls these snapshots from a library like Docker Hub and unpacks them so they are ready to use

APIs and CLI :

- Provides a way for you run the command like docker run which automates and the engine understands and runs it

Resource Management :

- It allocates CPU , Memory and storage to containers and making sure they don’t interfere with each other

Networking Setup :

- The engine sets up virtual networks so containers can talk to each other and the outside world

Storage Management :

- It handles volumes and data security so files don't disappear when a container stops

Therefore ,

- Container Engine is the middle ware that takes your container images and works with the host OS and turns into a running separated and manageable container

Container : Container Engine Layer : Working :

- Container Engine is the layer that sits between the Host OS (Infrastructure) and the Bins/Lib and Applications
- Container Engine just talks directly to the computers existing operating system
- Container Engine is the brain that sits between your applications and physical hosts operating system
- Container Engine allows applications to share the same Operating System making them faster and lighter
- Container Engine talks to the Host OS and it asks the OS for resources like CPU , Memory , Storage and networking
- Container Engine takes your application settings and its bins/libs and the files it needs to run and bundles them into a single file called an image
- Container Engine pulls the image like static files containing your application code and its tools from a library and uses them to build a container
- Container Engine can manage containers like it can start , stop , move and delete containers when needed
- Even though all the application share the same Operating System , the container engine uses namespaces and control groups which are the features of Linux Kernel to keep them separate
- Name spaces is where the application 1 thinks it's the only application on the computer cannot see or touch the files or processes of Application 2
- Control Groups ensures that application1 doesn't use all the RAM or CPU

- Container Engine shares the OS kernel where containers don't need a full guest OS and the container engine makes sure each container thinks its has own OS but they all share the host kernel

Therefore ,

- Without container engine you have to manually set up separated environments and manage resources and handle networking and the container engine automates all of this

Containers : App1 : Layer :

- App1 layer in container represents your application code with python script or java program or custom application
- In container this layer is the actual software you want to run like a web server , a database or custom application
- The App1 layer is like business logic of our project and not just code it's the entire environment required to function
- App1 layer in container represents the final executable software that you actually want to run

Containers : App1 : Inside :

- App 1 layer contains your actual code and settings you want which needs to run
- Your application code are files you wrote based on Python , Java or Html like app.py , index.html
- The application code is heart of the layer and the actual source code you wrote and its logic that performs the work
- The application code is also the actual instructions like Python , Java files that make the program run
- Dependencies are specific libraries or modules you application needs and they are not part of the main operating system like python packages
- Dependencies are also packages or modules you code calls like we use (Python) requirements.txt or package.json (Node.js)

- Environment Variables are the configuration details like database passwords like server port = 8080
- Environment Variables are specific instructions for your application such as settings like port=8080
- Environment Variables and configuration files that tell the application how to behave like which database to connect to

Containers : App1 : Working :

- The App 1 layer contains only the actual files and code needed to run that specific program
- If your app1 is a python web application then this layer contains your app.py file and your logic code
- In container a app1 thinks it is the only thing running on the computer where resources are provided by the Container Engine
- When app1 needs to do something like save a file or send data over the internet . It sends a request straight through the container engine to the Host Operating System kernel
- App1 is directly above Bins/Libs as the app1 relies on the layer below it to actually run .
- This Bins/Lib layer contains the operating system tools like basic file structures and dependencies like Python
- If app1 creates a log file or temporary image , it is stored in this app1 layer and you delete the container that layer app1 vanishes but the original app1 image remains safe and untouched

Comparison between Virtual Machines and Containers

- A virtual machine is like a complete computer inside your computer and it includes its own operating system like Guest OS and its own kernel and own libraries and own applications
- A container is a lightweight , package that contains everything an application needs to run like code , libraries , settings and dependencies where all in one place

Working of Virtual Machines and Containers :

Infrastructure Layer :

- (Virtual Machines) :Physical Hardware like CPU , RAM , Storage , Network
- (Containers) : Same Physical Hardware

Hypervisor Layer :

- (Virtual Machines) : Software that sits between hardware and VMs , it divides physical resources into virtual ones

Guest OS Layers :

- (Virtual Machine) :Each VM has its own full OS like Windows , Linux with its own kernel

Host OS Layers :

- (Containers) : One operating system like Linux runs on the hardware and its kernel is shared by all containers

Container Engine Layer :

- (Containers) : Software like Docker that manages containers and talks to the OS kernel

Bins/Lib Layer :

- (Virtual Machine) : Each VM has its own set of binaries and libraries like Python , Java
- (Container) : Each container has its own required binaries and libraries but only what the application needs

Application Layer :

- (Virtual Machine) : Your application runs inside this VM
- (Container) : Your application runs here

Key points : Virtual Machine and Container :

Virtual Machine :

- Each VM is isolated and thinks it has its own hardware
- The hypervisor manages resources and translates requests from Guest OS to physical hardware
- VMs are heavy because they include a full OS and duplicate libraries
- If one VM crashes other are not affected

Containers :

- Containers share the host OS kernel and no need for a Guest OS
- They use namespaces for isolation so containers don't see each other
- The use control groups to limit resource usage like CPU , RAM and etc
- Containers are lightweight and start in seconds and use less memory
- If containers crashes , other are not affected

Examples : Virtual Machine :

- You have a Windows Laptop and you install VMware and inside it you run Ubuntu and that Ubuntu is a VM and it has its own file system , apps and thinks its on real hardware

Containers :

- You have a Linux Laptop and you install docker and you run a python application in one container and a MySQL database in another.
- Both share the same Linux kernel but are separated from each other

Comparison between Virtual Machine Layers and Containers Layers :

Infrastructure Layer (The Physical Hardware) :

Virtual Machine :

- This is the actual physical hardware like CPU , RAM , Storage and Network cables
- The hypervisor sits directly on top of this layer

Container :

- Same physical hardware is used but containers don't interact with it directly
- The Host OS sits on top of this layer and containers share it

Therefore , both use the same physical hardware but VMs add a hypervisor while containers use the Host OS

Hypervisor Layer (Only in Virtual Machine) :

Virtual Machine :

- The hypervisor is a software layer that splits physical hardware into virtual versions
- It allows multiple Guest OS to run on one physical machine
- It manages CPU , Memory and isolation between VMs

Containers :

- No hypervisor in containers
- Containers don't have a Guest OS
- The Container Engine does this job

Therefore , VMs use a hypervisor and containers do not

Guest OS Layer (Only in VMs) :

Virtual Machine :

- Each VM has its own full Guest OS like Windows or Linux
- The Guest OS thinks its running on real hardware
- It includes its own kernel , drivers , file system and system services

Containers :

- No Guest OS in containers
- Containers share the Host OS kernel
- The Host OS provides the kernel for all containers

Therefore , VMs are heavy because each has its own OS . Containers are light because they share the Host OS

Container Engine Layer (Only in Containers) :

Virtual Machine :

- No container engine in VMs
- VMs use the hypervisor to manage resources

Container :

- The container engine like Docker sits on the Host OS
- It creates , starts , stops and manages containers
- It talks to the Host OS kernel to isolate containers using namespaces and groups

Therefore, Container Engine replaces the hypervisor and Guest OS in containers

Bins/Libs Layer :

Virtual Machine :

- Each VM has its own Bins/ Libs inside its Guest OS
- This makes VMs larger in size
- If 3 VMs run the same app , each has its own copy of libraries

Container :

- Containers share the Host OS bins/libs but can also have their own if needed
- This makes containers smalls in size
- If 3 containers run the same app they share most libraries

Therefore , VMs duplicates libraries and containers ahare them

App Layer :

- Virtual Machine :
- The app runs inside a VM with its own Guest OS and libraries
- The app is isolated at the VM level
- Slower to start because the OS must boot

Container :

- The app runs inside a container sharing the Host OS kernel
- The app is isolated at the process level using namespaces
- Starts in seconds because no OS boot is needed

Therefor both runs apps , but containers are faster and more portable

One powerful machine are run by these Virtual Machine and Containers :

Using Virtual Machines :

- One core reason is the hypervisor layer
- The hypervisor sits directly on the physical hardware (infrastructure layer) and creates virtual versions of the CPU , RAM and disk and network components
- Thus divides those physical resources into virtual ones
- Each Virtual machine gets to use physical CPU at a time to one VMs and then quickly to another making it feel like all are running continuously
- The hypervisor ensures that each VM and its Guest OS are completely separated and so that if one VMs crashes or gets virus , others VMs are safe

Using Containers :

- The core reason is the Shared Host Operating System Kernel
- Containers do not have their full guest OS
- So all containers on the hist use the same kernel of the Host OS
- Eliminates the need to bott and run a separate OS for every application making them lighter
- The Host OS kernel with the help from the container engine uses features like namespaces and control groups
- Namespaces creates a private space for each container making each think it has own separate environment
- Control groups limit how much CPU , memory or storage each container can use preventing one container from using all resources
- The container engine like Docker takes your application code and its dependencies and bundles them into an image and tells the Host OS kernel to start it as a container
- So it automates the process of sharing the OS and managing resources